Chapter 4. Networking

In this chapter

- 4.1 Connecting to a Server
- 4.2 Implementing Servers
- 4.3 Getting Web Data
- 4.4 The HTTP Client
- 4.5 Sending E-Mail

We begin this chapter by reviewing basic networking concepts, then move on to writing Java programs that connect to network services. We will show you how network clients and servers are implemented. Finally, you will see how to send e-mail from a Java program and how to harvest information from a web server.

4.1 Connecting to a Server

In the following sections, you will connect to a server, first by hand and with telnet, and then with a Java program.

4.1.1 Using Telnet

The telnet program is a great debugging tool for network programming. You should be able to launch it by typing `telnet` from a command shell.



Note

In Windows, you need to activate telnet. Go to the Control Panel, select Programs, click Turn Windows Features On or Off, and select the Telnet client checkbox. The Windows firewall also blocks quite a few network ports that we use in this chapter; you might need an administrator account to unblock them.

You may have used telnet to connect to a remote computer, but you can use it to communicate with other services provided by Internet hosts as well. Here is an example of what you can do. Type

```
telnet time-a.nist.gov 13
```

As **Figure 4.1** shows, you should get back a line like this:

Click here to view code image

```
57488 16-04-10 04:23:00 50 0 0 610.5 UTC(NIST) *
```



Figure 4.1 Output of the "time of day" service

What is going on? You have connected to the "time of day" service that most UNIX machines constantly run. The particular server that you connected to is operated by the National Institute of Standards and Technology and gives the measurement of a Cesium atomic clock. (Of course, the reported time is not completely accurate due to network delays.)

By convention, the "time of day" service is always attached to "port" number 13.

Note

In network parlance, a port is not a physical device, but an abstraction facilitating communication between a server and a client (see **Figure 4.2**).

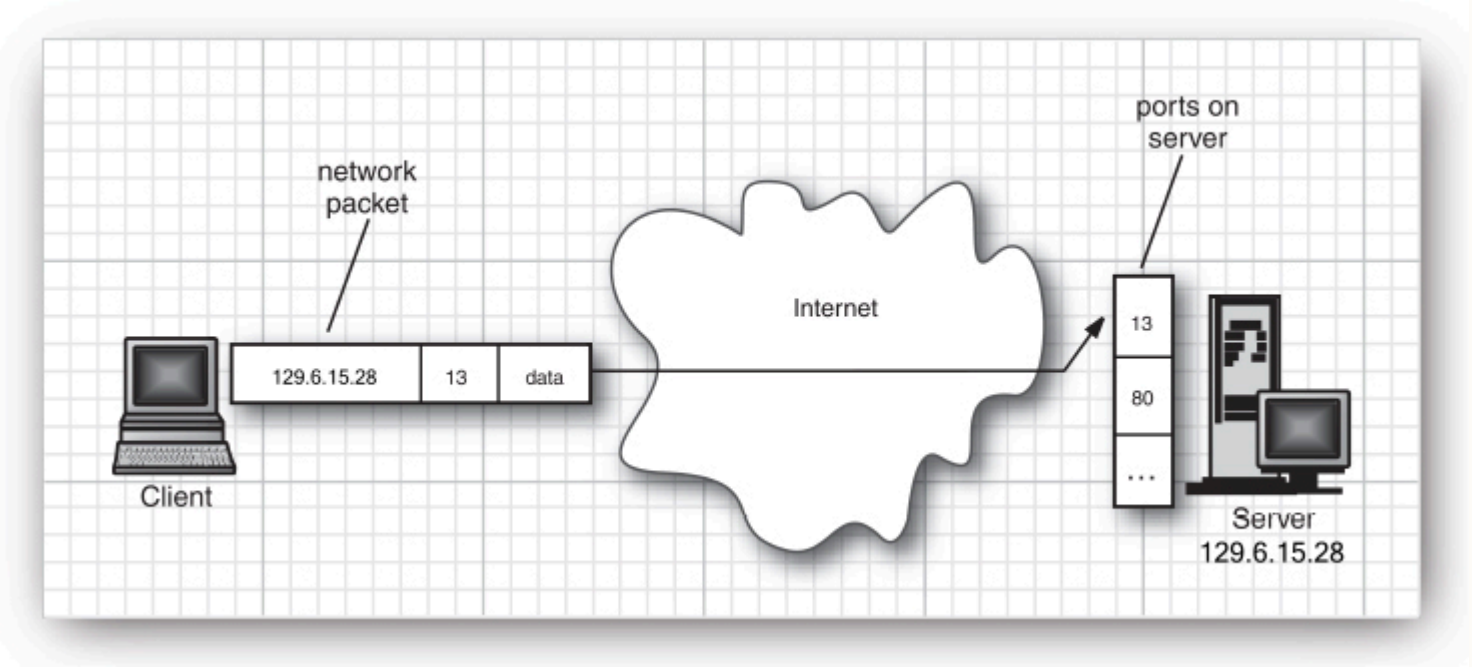


Figure 4.2 A client connecting to a server port

The server software is continuously running on the remote machine, waiting for any network traffic that wants to chat with port 13. When the operating system on the remote computer receives a network package that contains a request to connect to port

number 13, it wakes up the listening server process and establishes the connection. The connection stays up until it is terminated by one of the parties.

When you began the telnet session with `time-a.nist.gov` at port 13, a piece of network software knew enough to convert the string `"time-a.nist.gov"` to its correct Internet Protocol (IP) address, 129.6.15.28. The telnet software then sent a connection request to that address, asking for a connection to port 13. Once the connection was established, the remote program sent back a line of data and closed the connection. In general, of course, clients and servers engage in a more extensive dialog before one or the other closes the connection.

Here is another experiment along the same lines—but a bit more interesting. Type

```
telnet horstmann.com 80
```

Then type very carefully the following:

```
GET / HTTP/1.1
Host: horstmann.com
blank line
```

That is, hit the Enter key twice at the end.

Figure 4.3 shows the response. It should look eerily familiar—you got a page of HTML-formatted text, namely Cay Horstmann's home page.



Figure 4.3 Using telnet to access an HTTP port

This is exactly the same process that your web browser goes through to get a web page. It uses HTTP to request web pages from servers. Of course, the browser displays the HTML code more nicely.



Note

The `Host` key/value pair is required when you connect to a web server that hosts multiple domains at the same IP address. You can omit it if the server hosts a single domain.

4.1.2 Connecting to a Server with Java

Our first network program in **Listing 4.1** will do the same thing we did using telnet—connect to a port and print out what it finds.

Listing 4.1 socket/SocketTest.java

Click here to view code image

```
 1  package socket;
 2
 3  import java.io.*;
 4  import java.net.*;
 5  import java.nio.charset.*;
 6  import java.util.*;
 7
 8  /**
 9   * This program makes a socket connection to the atomic clock in Boulder, Colorado, and
10   * prints the time that the server sends.
11   * @version 1.22 2018-03-17
12   * @author Cay Horstmann
13   */
14  public class SocketTest
15  {
16     public static void main(String[] args) throws IOException
17     {
18        try (var s = new Socket("time-a.nist.gov", 13);
```

```
19              var in = new Scanner(s.getInputStream(), StandardCharsets.UTF_8))
20        {
21           while (in.hasNextLine())
22           {
23              String line = in.nextLine();
24              System.out.println(line);
25           }
26        }
27     }
28 }
```

The key statements of this simple program are these:

Click here to view code image

```
var s = new Socket("time-a.nist.gov", 13);
InputStream inStream = s.getInputStream();
```

The first line opens a *socket*, which is a network software abstraction that enables communication out of and into this program. We pass the remote address and the port number to the socket constructor. If the connection fails, an `UnknownHostException` is thrown. If there is another problem, an `IOException` occurs. Since `UnknownHostException` is a subclass of `IOException` and this is a sample program, we just catch the superclass.

Once the socket is open, the `getInputStream` method in `java.net.Socket` returns an `InputStream` object that you can use just like any other stream. Once you have grabbed the stream, this program simply prints each input line to standard output. This process continues until the stream is finished and the server disconnects.

This program works only with very simple servers, such as a "time of day" service. In more complex networking programs, the client sends request data to the server, and the server might not immediately disconnect at the end of a response. You will see how to implement that behavior in several examples throughout this chapter.

The `Socket` class is pleasant and easy to use because the Java library hides the complexities of establishing a networking connection and sending data across it. The `java.net` package essentially gives you the same programming interface you would use to work with a file.



Note

In this book, we cover only the Transmission Control Protocol (TCP). The Java platform also supports the User Datagram Protocol (UDP), which can be used to send packets (also called *datagrams*) with much less overhead than TCP. The drawback is that packets need not be delivered in sequential order to the receiving application and can even be dropped altogether. It is up to the recipient to put the packets in order and to request retransmission of missing packets. UDP is well suited for applications in which missing packets can be tolerated—for example, for audio or video streams or continuous measurements.

`java.net.Socket` **1.0**

- `Socket(String host, int port)`
  constructs a socket to connect to the given host and port.
- `InputStream getInputStream()`
- `OutputStream getOutputStream()`
  get the stream to read data from the socket or write data to the socket.

4.1.3 Socket Timeouts

Reading from a socket blocks until data are available. If the host is unreachable, your application waits for a long time and you are at the mercy of the underlying operating system to eventually time out.

You can decide what timeout value is reasonable for your particular application. Then, call the `setSoTimeout` method to set a timeout value (in milliseconds).

Click here to view code image

```
var s = new Socket(. . .);
s.setSoTimeout(10000); // time out after 10 seconds
```

If the timeout value has been set for a socket, all subsequent read operations throw a `SocketTimeoutException` when the timeout has been reached before the operation has completed its work. You can catch that exception and react to the timeout.

Click here to view code image

```
try
{
  InputStream in = s.getInputStream(); // read from in
  . . .
}
catch (SocketTimeoutException e)
{
```

```
   react to timeout
}
```

There is no timeout for write operations.

There is one additional timeout issue that you need to address. The constructor

```
Socket(String host, int port)
```

can block indefinitely until an initial connection to the host is established.

You can overcome this problem by first constructing an unconnected socket and then connecting it with a timeout:

Click here to view code image

```
var s = new Socket();
s.connect(new InetSocketAddress(host, port), timeout);
```

See Section 4.2.4, "Interruptible Sockets," on p. 252 for how to allow users to interrupt a socket connection at any time.

java.net.Socket 1.0

- Socket() 1.1
  creates a socket that has not yet been connected.
- void connect(SocketAddress address) 1.4
  connects this socket to the given address.
- void connect(SocketAddress address, int timeoutInMilliseconds) 1.4
  connects this socket to the given address, or returns if the time interval expired.
- void setSoTimeout(int timeoutInMilliseconds) 1.1
  sets the blocking time for read requests on this socket. If the timeout is reached, a SocketTimeoutException is raised.
- boolean isConnected() 1.4
  returns true if the socket is connected.
- boolean isClosed() 1.4
  returns true if the socket is closed.

4.1.4 Internet Addresses

Usually, you don't have to worry too much about Internet addresses—the numerical host addresses that consist of 4 bytes (or, with IPv6, 16 bytes) such as 129.6.15.28. However, you can use the InetAddress class if you need to convert between host names and Internet addresses.

The java.net package supports IPv6 Internet addresses, provided the host operating system does.

The static getByName method returns an InetAddress object of a host. For example,

Click here to view code image

```
InetAddress address = InetAddress.getByName("time-a.nist.gov");
```

returns an `InetAddress` object that encapsulates the sequence of four bytes 129.6.15.28. You can access the bytes with the `getAddress` method.

Click here to view code image

```
byte[] addressBytes = address.getAddress();
```

Some host names with a lot of traffic correspond to multiple Internet addresses, to facilitate load balancing. For example, at the time of this writing, the host name **google.com** corresponds to twelve different Internet addresses. One of them is picked at random when the host is accessed. You can get all hosts with the `getAllByName` method.

Click here to view code image

```
InetAddress[] addresses = InetAddress.getAllByName(host);
```

Finally, you sometimes need the address of the local host. If you simply ask for the address of `localhost`, you always get the local loopback address 127.0.0.1, which cannot be used by others to connect to your computer. Instead, use the static `getLocalHost` method to get the address of your local host.

Click here to view code image

```
InetAddress address = InetAddress.getLocalHost();
```

Listing 4.2 is a simple program that prints the Internet address of your local host if you do not specify any command-line parameters, or all Internet addresses of another host if you specify the host name on the command line, such as

Click here to view code image

```
java inetAddress/InetAddressTest www.horstmann.com
```

Listing 4.2 inetAddress/InetAddressTest.java

Click here to view code image

```
1  package inetAddress;
2
3  import java.io.*;
4  import java.net.*;
5
6  /**
7   * This program demonstrates the InetAddress class. Supply a host name as command-line
8   * argument, or run without command-line arguments to see the address of the local host.
9   * @version 1.02 2012-06-05
```

```
10   * @author Cay Horstmann
11   */
12  public class InetAddressTest
13  {
14     public static void main(String[] args) throws IOException
15     {
16        if (args.length > 0)
17        {
18           String host = args[0];
19           InetAddress[] addresses = InetAddress.getAllByName(host);
20           for (InetAddress a : addresses)
21           System.out.println(a);
22        }
23        else
24        {
25           InetAddress localHostAddress = InetAddress.getLocalHost();
26           System.out.println(localHostAddress);
27        }
28     }
29  }
```

java.net.InetAddress 1.0

- `static InetAddress getByName(String host)`
- `static InetAddress[] getAllByName(String host)`
  construct an `InetAddress`, or an array of all Internet addresses, for the given host name.
- `static InetAddress getLocalHost()`
  constructs an `InetAddress` for the local host.
- `byte[] getAddress()`

returns an array of bytes that contains the numerical address.

- `String getHostAddress()`
  returns a string with decimal numbers, separated by periods, for example `"129.6.15.28"`.
- `String getHostName()`
  returns the host name.

4.2 Implementing Servers

Now that we have implemented a basic network client that receives data from the Internet, let's program a simple server that can send information to clients.

4.2.1 Server Sockets

A server program, when started, waits for a client to attach to its port. For our example program, we chose port number 8189, which is not used by any of the standard services. The `ServerSocket` class establishes a socket. In our case, the command

Click here to view code image

```
var s = new ServerSocket(8189);
```

establishes a server that monitors port 8189. The command

```
Socket incoming = s.accept();
```

tells the program to wait indefinitely until a client connects to that port. Once someone connects to this port by sending the correct request over the network, this method returns a `Socket` object that represents the connection that was made. You can use this object to get input and output streams, as is shown in the following code:

Click here to view code image

```
InputStream inStream = incoming.getInputStream();
OutputStream outStream = incoming.getOutputStream();
```

Everything that the server sends to the server output stream becomes the input of the client program, and all the output from the client program ends up in the server input stream.

In all the examples in this chapter, we transmit text through sockets. We therefore turn the streams into scanners and writers.

Click here to view code image

```
var in = new Scanner(inStream, StandardCharsets.UTF_8);
var out = new PrintWriter(new OutputStreamWriter(outStream, StandardCharsets.UTF_8),
    true /* autoFlush */);
```

Let's send the client a greeting:

Click here to view code image

```
out.println("Hello! Enter BYE to exit.");
```

When you use telnet to connect to this server program at port 8189, you will see this greeting on the terminal screen.

In this simple server, we just read the client's input, a line at a time, and echo it. This demonstrates that the program receives the input. An actual server would obviously compute and return an answer depending on the input.

Click here to view code image

```
String line = in.nextLine();
out.println("Echo: " + line);
if (line.strip().equals("BYE")) done = true;
```

In the end, we close the incoming socket.

```
incoming.close();
```

That is all there is to it. Every server program, such as an HTTP web server, continues performing this loop:

1. It receives a command from the client ("get me this information") through an incoming data stream.
2. It decodes the client command.

3. It gathers the information that the client requested.
4. It sends the information to the client through the outgoing data stream.

Listing 4.3 is the complete program.

Listing 4.3 server/EchoServer.java

Click here to view code image

```
 1  package server;
 2
 3  import java.io.*;
 4  import java.net.*;
 5  import java.nio.charset.*;
 6  import java.util.*;
 7
 8  /**
 9   * This program implements a simple server that listens to port 8189 and echoes back all
10   * client input.
11   * @version 1.22 2018-03-17
12   * @author Cay Horstmann
13   */
14  public class EchoServer
15  {
16     public static void main(String[] args) throws IOException
17     {
18        // establish server socket
19        try (var s = new ServerSocket(8189))
20        {
21           // wait for client connection
```

```
22          try (Socket incoming = s.accept())
23          {
24             InputStream inStream = incoming.getInputStream();
25             OutputStream outStream = incoming.getOutputStream();
26
27             try (var in = new Scanner(inStream, StandardCharsets.UTF_8))
28             {
29                var out = new PrintWriter(
30                  new OutputStreamWriter(outStream, StandardCharsets.UTF_8),
31                  true /* autoFlush */);
32
33                out.println("Hello! Enter BYE to exit.");
34
35                // echo client input
36                boolean done = false;
37                while (!done && in.hasNextLine())
38                {
39                   String line = in.nextLine();
40                   out.println("Echo: " + line);
41                   if (line.strip().equals("BYE")) done = true;
42                }
43             }
44          }
45       }
46    }
47 }
```

To try it out, compile and run the program. Then use telnet to connect to the server `localhost` (or IP address 127.0.0.1) and port 8189.

If you are connected directly to the Internet, anyone in the world can access your echo server, provided they know your IP address and the magic port number.

When you connect to the port, you will see the message shown in **Figure 4.4**:

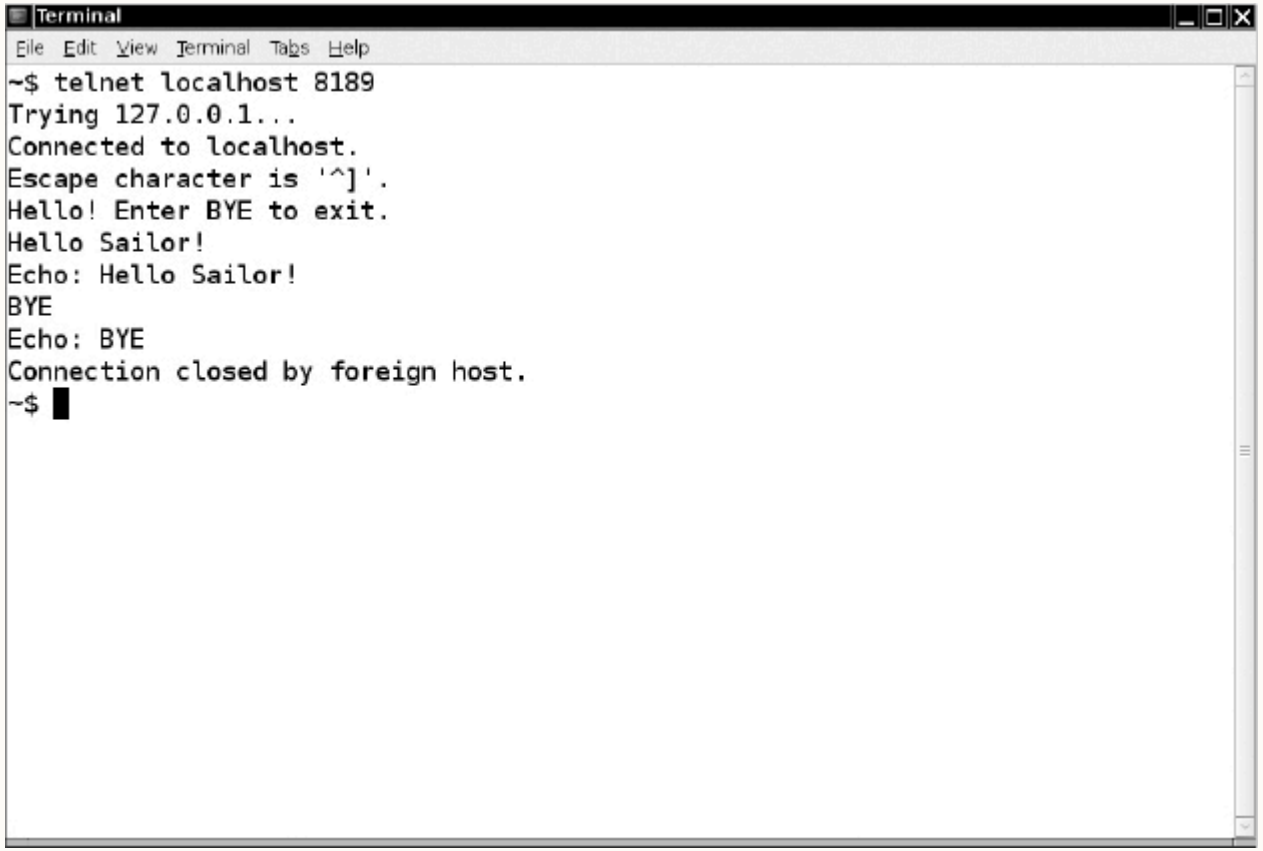


Figure 4.4 Accessing an echo server

```
Hello! Enter BYE to exit.
```

Type anything and watch the input echo on your screen. Type `BYE` (all uppercase letters) to disconnect. The server program will terminate as well.

`java.net.ServerSocket` **1.0**

- `ServerSocket(int port)`
  creates a server socket that monitors a port.
- `Socket accept()`
  waits for a connection. This method blocks (i.e., idles) the current thread until the connection is made. The method returns a `Socket` object through which the program can communicate with the connecting client.
- `void close()`
  closes the server socket.

4.2.2 Serving Multiple Clients

There is one problem with the simple server in the preceding example. Suppose we want to allow multiple clients to connect to our server at the same time. Typically, a server runs constantly on a server computer, and clients from all over the Internet might want to use it at the same time. Without support for multiple connections, any one client can monopolize the service by connecting to it for a long time. We can do much better through the magic of threads.

Every time we know the program has established a new socket connection—that is, every time the call to `accept()` returns a socket—we will launch a new thread to take care of the connection between the server and *that* client. The main program will just

go back and wait for the next connection. For this to happen, the main loop of the server should look like this:

Click here to view code image

```
while (true)
{
  Socket incoming = s.accept();
  var r = new ThreadedEchoHandler(incoming);
  var t = new Thread(r);
  t.start();
}
```

The `ThreadedEchoHandler` class implements `Runnable` and contains the communication loop with the client in its `run` method.

Click here to view code image

```
class ThreadedEchoHandler implements Runnable
{
  . . .
  public void run()
  {
    try (InputStream inStream = incoming.getInputStream();
         OutputStream outStream = incoming.getOutputStream())
    {
      Process input and send response
    }
    catch(IOException e)
    {
```

```
        Handle exception
      }
    }
  }
```

When each connection starts a new thread, multiple clients can connect to the server at the same time. You can easily check this out.

1. Compile and run the server program (**Listing 4.4**).
2. Open several telnet windows as we have in **Figure 4.5**.

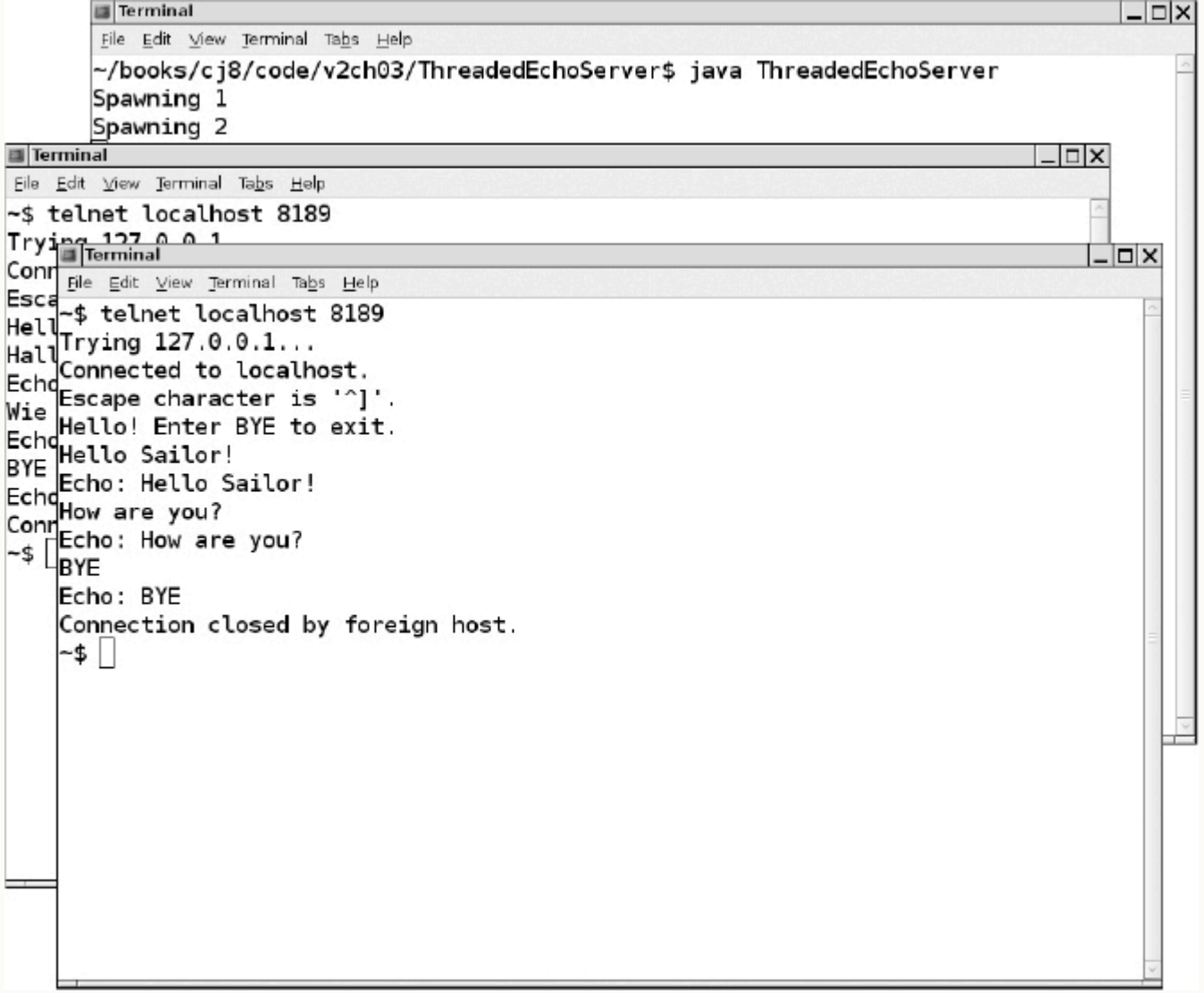


Figure 4.5 Several telnet windows communicating simultaneously

3. Switch between windows and type commands. Note that you can communicate through all of them simultaneously.

4. When done, switch to the window from which you launched the server program and press Ctrl+C to kill it.



Note

In this program, we spawn a separate thread for each connection. This approach is not satisfactory for high-performance servers. You can achieve greater server throughput by using features of the `java.nio` package. See **www.ibm.com/developerworks/java/library/j-javaio** for more information.

Listing 4.4 threaded/ThreadedEchoServer.java

Click here to view code image

```
 1  package threaded;
 2
 3  import java.io.*;
 4  import java.net.*;
 5  import java.nio.charset.*;
 6  import java.util.*;
 7
 8  /**
 9   * This program implements a multithreaded server that listens to port 8189 and echoes back
10   * all client input.11   * @author Cay Horstmann
12   * @version 1.23 2018-03-17
13   */
14  public class ThreadedEchoServer
15  {
16     public static void main(String[] args )
17     {
```

```
18        try (var s = new ServerSocket(8189))
19        {
20          int i = 1;
21
22          while (true)
23          {
24             Socket incoming = s.accept();
25             System.out.println("Spawning " + i);
26             Runnable r = new ThreadedEchoHandler(incoming);
27             var t = new Thread(r);
28             t.start();
29             i++;
30          }
31        }
32        catch (IOException e)
33        {
34           e.printStackTrace();
35        }
36     }
37  }
38
39  /**
40   * This class handles the client input for one server socket connection.
41   */
42   class ThreadedEchoHandler implements Runnable
43   {
44      private Socket incoming;
45
46   /**
47     Constructs a handler.
48     @param incomingSocket the incoming socket
49    */
50    public ThreadedEchoHandler(Socket incomingSocket)
```

```
51    {
52       incoming = incomingSocket;
53    }
54
55    public void run()
56    {
57       try (InputStream inStream = incoming.getInputStream();58            OutputStream outStre
59            var in = new Scanner(inStream, StandardCharsets.UTF_8);
60            var out = new PrintWriter(
61                new OutputStreamWriter(outStream, StandardCharsets.UTF_8),
62                true /* autoFlush */))
63       {
64         out.println( "Hello! Enter BYE to exit." );
65
66         // echo client input
67         boolean done = false;
68         while (!done && in.hasNextLine())
69         {
70            String line = in.nextLine();
71            out.println("Echo: " + line);
72            if (line.strip().equals("BYE"))
73               done = true;
74         }
75       }
76       catch (IOException e)
77       {
78          e.printStackTrace();
79       }
80    }
81 }
```

4.2.3 Half-Close

The *half-close* allows one end of a socket connection to terminate its output while still receiving data from the other end.

Here is a typical situation. Suppose you transmit data to the server but you don't know at the outset how much data you have. With a file, you'd just close the file at the end of the data. However, if you close a socket, you immediately disconnect from the server and cannot read the response.

The half-close solves this problem. You can close the output stream of a socket, thereby indicating to the server the end of the requested data, but keep the input stream open.

The client side looks like this:

Click here to view code image

```
try (var socket = new Socket(host, port))
{
  var in = new Scanner(socket.getInputStream(), StandardCharsets.UTF_8);
  var writer = new PrintWriter(socket.getOutputStream());
  // send request data
  writer.print(. . .);
  writer.flush();
  socket.shutdownOutput();
  // now socket is half-closed
  // read response data
  while (in.hasNextLine() != null)
  {
    String line = in.nextLine();
    . . .
```

```
    }
  }
```

The server side simply reads input until the end of the input stream is reached. Then it sends the response.

Of course, this protocol is only useful for one-shot services such as HTTP where the client connects, issues a request, catches the response, and then disconnects.

`java.net.Socket` **1.0**

- `void shutdownOutput()` **1.3**
  sets the output stream to "end of stream."
- `void shutdownInput()` **1.3**
  sets the input stream to "end of stream."
- `boolean isOutputShutdown()` **1.4**
  returns `true` if output has been shut down.
- `boolean isInputShutdown()` **1.4**
  returns `true` if input has been shut down.

4.2.4 Interruptible Sockets

When you connect to a socket, the current thread blocks until the connection has been established or a timeout has elapsed. Similarly, when you read data through a socket, the current thread blocks until the operation is successful or has timed out. (There is no timeout for writing data.)

In interactive applications, you would like to give users an option to simply cancel a socket connection that does not appear to produce results. However, if a thread blocks on an unresponsive socket, you cannot unblock it by calling `interrupt`.

To interrupt a socket operation, use a `SocketChannel`, a feature of the `java.nio` package. Open the `SocketChannel` like this:

Click here to view code image

```
SocketChannel channel = SocketChannel.open(new InetSocketAddress(host, port));
```

A channel does not have associated streams. Instead, it has `read` and `write` methods that make use of `Buffer` objects. (See Chapter 1 for more information about NIO buffers.) These methods are declared in the interfaces `ReadableByteChannel` and `WritableByteChannel`.

If you don't want to deal with buffers, you can use the `Scanner` class to read from a `SocketChannel` because `Scanner` has a constructor with a `ReadableByteChannel` parameter:

Click here to view code image

```
var in = new Scanner(channel, StandardCharsets.UTF_8);
```

To turn a channel into an output stream, use the static `Channels.newOutputStream` method.

Click here to view code image

```
OutputStream outStream = Channels.newOutputStream(channel);
```

That's all you need to do. Whenever a thread is interrupted during an open, read, or write operation, the operation does not block, but is terminated with an exception.

The program in **Listing 4.5** contrasts interruptible and blocking sockets. A server sends numbers and pretends to be stuck after the tenth number. Click on either button, and a thread is started that connects to the server and prints the output. The first thread uses an interruptible socket; the second thread uses a blocking socket. If you click the Cancel button within the first ten numbers, you can interrupt either thread.

However, after the first ten numbers, you can only interrupt the first thread. The second thread keeps blocking until the server finally closes the connection (see **Figure 4.6**).



Figure 4.6 Interrupting a socket

Listing 4.5 interruptible/InterruptibleSocketTest.java

Click here to view code image

```
 1  package interruptible;
 2
 3  import java.awt.*;
 4  import java.awt.event.*;
 5  import java.util.*;
 6  import java.net.*;
 7  import java.io.*;
 8  import java.nio.charset.*;
 9  import java.nio.channels.*;
10  import javax.swing.*;
11
12  /**
13   * This program shows how to interrupt a socket channel.
14   * @author Cay Horstmann
15   * @version 1.05 2018-03-17
16   */
17  public class InterruptibleSocketTest
18  {
19     public static void main(String[] args)
20     {
```

```
21       EventQueue.invokeLater(() ->
22          {
23             var frame = new InterruptibleSocketFrame();
24             frame.setTitle("InterruptibleSocketTest");
25             frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
26             frame.setVisible(true);
27          });
28    }
29 }
30
31 class InterruptibleSocketFrame extends JFrame
32 {
33     private Scanner in;
34     private JButton interruptibleButton;
35     private JButton blockingButton;
36     private JButton cancelButton;
37     private JTextArea messages;
38     private TestServer server;
39     private Thread connectThread;
40
41     public InterruptibleSocketFrame()
42     {
43        var northPanel = new JPanel();
44        add(northPanel, BorderLayout.NORTH);
45
46        final int TEXT_ROWS = 20;
47        final int TEXT_COLUMNS = 60;
48        messages = new JTextArea(TEXT_ROWS, TEXT_COLUMNS);
49        add(new JScrollPane(messages));50
51        interruptibleButton = new JButton("Interruptible");
52        blockingButton = new JButton("Blocking");
53
54        northPanel.add(interruptibleButton);
```

```
55          northPanel.add(blockingButton);
56
57          interruptibleButton.addActionListener(event ->
58             {
59                interruptibleButton.setEnabled(false);
60                blockingButton.setEnabled(false);
61                cancelButton.setEnabled(true);
62                connectThread = new Thread(() ->
63                   {
64                      try
65                      {
66                         connectInterruptibly();
67                      }
68                      catch (IOException e)
69                      {
70                         messages.append("\nInterruptibleSocketTest.connectInterruptibly: " + e
71                      }
72                   });
73                connectThread.start();
74             });
75
76          blockingButton.addActionListener(event ->
77             {
78                interruptibleButton.setEnabled(false);
79                blockingButton.setEnabled(false);
80                cancelButton.setEnabled(true);
81                connectThread = new Thread(() ->
82                   {
83                      try
84                      {
85                         connectBlocking();
86                      }
87                      catch (IOException e)
```

```
88                  {
89                      messages.append("\nInterruptibleSocketTest.connectBlocking: " + e);
90                  }
91               });
92            connectThread.start();
93         });
94
95       cancelButton = new JButton("Cancel");
96       cancelButton.setEnabled(false);
97       northPanel.add(cancelButton);
98       cancelButton.addActionListener(event ->99           {
100            connectThread.interrupt();
101            cancelButton.setEnabled(false);
102         });
103      server = new TestServer();
104      new Thread(server).start();
105      pack();
106   }
107
108   /**
109    * Connects to the test server, using interruptible I/O
110    */
111   public void connectInterruptibly() throws IOException
112   {
113      messages.append("Interruptible:\n");
114      try (SocketChannel channel
115            = SocketChannel.open(new InetSocketAddress("localhost", 8189)))
116      {
117        in = new Scanner(channel, StandardCharsets.UTF_8);
118        while (!Thread.currentThread().isInterrupted())
119        {
120           messages.append("Reading ");
121           if (in.hasNextLine())
```

```
122             {
123                String line = in.nextLine();
124                messages.append(line);
125                messages.append("\n");
126             }
127          }
128       }
129       finally
130       {
131          EventQueue.invokeLater(() ->
132             {
133                messages.append("Channel closed\n");
134                interruptibleButton.setEnabled(true);
135                blockingButton.setEnabled(true);
136             });
137       }
138    }
139
140    /**
141     * Connects to the test server, using blocking I/O
142     */
143    public void connectBlocking() throws IOException
144    {
145       messages.append("Blocking:\n");146       try (var sock = new Socket("localhost", 8189))
147       {
148          in = new Scanner(sock.getInputStream(), StandardCharsets.UTF_8);
149          while (!Thread.currentThread().isInterrupted())
150          {
151              messages.append("Reading ");
152              if (in.hasNextLine())
153              {
154                 String line = in.nextLine();
155                 messages.append(line);
```

```
156                messages.append("\n");
157             }
158          }
159       }
160       finally
161       {
162          EventQueue.invokeLater(() ->
163             {
164                messages.append("Socket closed\n");
165                interruptibleButton.setEnabled(true);
166                blockingButton.setEnabled(true);
167             });
168       }
169    }
170
171    /**
172     * A multithreaded server that listens to port 8189 and sends numbers to the client,
173     * simulating a hanging server after 10 numbers.
174     */
175    class TestServer implements Runnable
176    {
177       public void run()
178       {
179          try (var s = new ServerSocket(8189))
180          {
181             while (true)
182             {
183                Socket incoming = s.accept();
184                Runnable r = new TestServerHandler(incoming);
185                new Thread(r).start();
186             }
187          }
188          catch (IOException e)
```

```
189         {
190            messages.append("\nTestServer.run: " + e);
191         }
192      }
193   }
194195   /**
196    * This class handles the client input for one server socket connection.
197    */
198   class TestServerHandler implements Runnable
199   {
200      private Socket incoming;
201      private int counter;
202
203      /**
204       * Constructs a handler.
205       * @param i the incoming socket
206       */
207      public TestServerHandler(Socket i)
208      {
209         incoming = i;
210      }
211
212      public void run()
213      {
214         try
215         {
216            try
217            {
218               OutputStream outStream = incoming.getOutputStream();
219               var out = new PrintWriter(
220                  new OutputStreamWriter(outStream, StandardCharsets.UTF_8),
221                  true /* autoFlush */);
222               while (counter < 100)
```

```
223                {
224                   counter++;
225                   if (counter <= 10) out.println(counter);
226                   Thread.sleep(100);
227                }
228             }
229             finally
230             {
231                incoming.close();
232                messages.append("Closing server\n");
233             }
234          }
235          catch (Exception e)
236          {
237             messages.append("\nTestServerHandler.run: " + e);
238          }
239       }
240    }
241 }
```

java.net.InetSocketAddress 1.4

- `InetSocketAddress(String hostname, int port)`
  constructs an address object with the given host and port, resolving the host name during construction. If the host name cannot be resolved, the address object's `unresolved` property is set to `true`.
- `boolean isUnresolved()`
  returns `true` if this address object could not be resolved.

java.nio.channels.SocketChannel 1.4

- `static SocketChannel open(SocketAddress address)`
  opens a socket channel and connects it to a remote address.

---

java.nio.channels.Channels 1.4

- `static InputStream newInputStream(ReadableByteChannel channel)`
  constructs an input stream that reads from the given channel.
- `static OutputStream newOutputStream(WritableByteChannel channel)`
  constructs an output stream that writes to the given channel.

4.3 Getting Web Data

To access web servers in a Java program, you will want to work at a higher level than socket connections and HTTP requests. In the following sections, we discuss the classes that the Java library provides for this purpose.

4.3.1 URLs and URIs

The `URL` and `URLConnection` classes encapsulate much of the complexity of retrieving information from a remote site. You can construct a `URL` object from a string:

```
var url = new URL(urlString);
```

If you simply want to fetch the contents of the resource, use the `openStream` method of the `URL` class. This method yields an `InputStream` object. Use it in the usual way—for example, to construct a `Scanner`:

Click here to view code image

```
InputStream inStream = url.openStream();
var in = new Scanner(inStream, StandardCharsets.UTF_8);
```

The `java.net` package makes a useful distinction between URLs (uniform resource *locators*) and URIs (uniform resource *identifiers*).

A URI is a purely syntactical construct that contains the various parts of the string specifying a web resource. A URL is a special kind of URI, namely one with sufficient information to *locate* a resource. Other URIs, such as

```
mailto:cay@horstmann.com
```

are not locators—there is no data to locate from this identifier. Such a URI is called a URN (uniform resource *name*).

In the Java library, the `URI` class has no methods for accessing the resource that the identifier specifies—its sole purpose is parsing. In contrast, the `URL` class can open a stream to the resource. For that reason, the `URL` class only works with schemes that the Java library knows how to handle, such as `http:`, `https:`, `ftp:`, the local file system (`file:`), and JAR files (`jar:`).

To see why parsing is not trivial, consider how complex URIs can be. For example,

Click here to view code image

```
http:/google.com?q=Beach+Chalet
ftp://username:password@ftp.yourserver.com/pub/file.txt
```

The URI specification gives the rules for the makeup of these identifiers. A URI has the syntax

```
[scheme:]schemeSpecificPart[#fragment]
```

Here, the `[. . .]` denotes an optional part, and the `:` and `#` are included literally in the identifier.

If the *scheme*: part is present, the URI is called *absolute*. Otherwise, it is called *relative*.

An absolute URI is *opaque* if the *schemeSpecificPart* does not begin with a `/` such as

```
mailto:cay@horstmann.com
```

All absolute nonopaque URIs and all relative URIs are *hierarchical*. Examples are

Click here to view code image

```
http://horstmann.com/index.html
../../java/net/Socket.html#Socket()
```

The `schemeSpecificPart` of a hierarchical URI has the structure

```
[//authority][path][?query]
```

where, again, `[. . .]` denotes optional parts.

For server-based URIs, the *authority* part has the form

```
[user-info@]host[:port]
```

The `port` must be an integer.

RFC 2396, which standardizes URIs, also supports a registry-based mechanism in which the *authority* has a different format, but this is not in common use.

One of the purposes of the `URI` class is to parse an identifier and break it up into its components. You can retrieve them with the methods

```
getScheme
getSchemeSpecificPart
getAuthority
getUserInfo
getHost
getPort
getPath
getQuery
getFragment
```

The other purpose of the `URI` class is the handling of absolute and relative identifiers. If you have an absolute URI such as

Click here to view code image

```
http://docs.mycompany.com/api/java/net/ServerSocket.html
```

and a relative URI such as

Click here to view code image

```
../../java/net/Socket.html#Socket()
```

then you can combine the two into an absolute URI.

Click here to view code image

```
http://docs.mycompany.com/api/java/net/Socket.html#Socket()
```

This process is called *resolving* a relative URL.

The opposite process is called *relativization*. For example, suppose you have a *base* URI

Click here to view code image

```
http://docs.mycompany.com/api
```

and a URI

Click here to view code image

```
http://docs.mycompany.com/api/java/lang/String.html
```

Then the relativized URI is

```
java/lang/String.html
```

The URI class supports both of these operations:

Click here to view code image

```
relative = base.relativize(combined);
combined = base.resolve(relative);
```

4.3.2 Using a `URLConnection` to Retrieve Information

If you want additional information about a web resource, you should use the `URLConnection` class, which gives you much more control than the basic `URL` class.

When working with a `URLConnection` object, you must carefully schedule your steps.

1. Call the `openConnection` method of the `URL` class to obtain the `URLConnection` object:
   Click here to view code image

   ```
   URLConnection connection = url.openConnection();
   ```

2. Set any request properties, using the methods

```
setDoInput
setDoOutput
setIfModifiedSince
setUseCaches
setAllowUserInteraction
setRequestProperty
setConnectTimeout
setReadTimeout
```

We discuss these methods later in this section and in the API notes.

3. Connect to the remote resource by calling the `connect` method:

```
connection.connect();
```

Besides making a socket connection to the server, this method also queries the server for *header information.*

4. After connecting to the server, you can query the header information. Two methods, `getHeaderFieldKey` and `getHeaderField`, enumerate all fields of the header. The method `getHeaderFields` gets a standard `Map` object containing the header fields. For your convenience, the following methods query standard fields:

```
getContentType
getContentLength
getContentEncoding
getDate
getExpiration
getLastModified
```

5. Finally, you can access the resource data. Use the `getInputStream` method to obtain an input stream for reading the information. (This is the same input stream that the `openStream` method of the `URL` class returns.) The other method, `getContent`, isn't very useful in practice. The objects that are returned by standard content types such as `text/plain` and `image/gif` require classes in the `com.sun` hierarchy for processing. You could register your own content handlers, but we do not discuss this technique in our book.



Caution

Some programmers form a wrong mental image when using the `URLConnection` class, thinking that the `getInputStream` and `getOutputStream` methods are similar to those of the `Socket` class. But that isn't quite true. The `URLConnection` class does quite a bit of magic behind the scenes—in particular, the handling of request and response headers. For that reason, it is important that you follow the setup steps for the connection.

Let us now look at some of the `URLConnection` methods in detail. Several methods set properties of the connection before connecting to the server. The most important ones are `setDoInput` and `setDoOutput`. By default, the connection yields an input stream for reading from the server but no output stream for writing. If you want an output stream (for example, for posting data to a web server), you need to call

```
connection.setDoOutput(true);
```

Next, you may want to set some of the request headers. The request headers are sent together with the request command to the server. Here is an example:

Click here to view code image

```
GET www.server.com/index.html HTTP/1.0
Referer: http://www.somewhere.com/links.html
Proxy-Connection: Keep-Alive
User-Agent: Mozilla/5.0 (X11; U; Linux i686; en-US; rv:1.8.1.4)
Host: www.server.com
Accept: text/html, image/gif, image/jpeg, image/png, */*
Accept-Language: en
Accept-Charset: iso-8859-1,*,utf-8
Cookie: orangemilano=192218887821987
```

The `setIfModifiedSince` method tells the connection that you are only interested in data modified since a certain date.

Finally, you can use the catch-all `setRequestProperty` method to set any name/value pair that is meaningful for the particular protocol. For the format of the HTTP request headers, see RFC 2616. Some of these parameters are not well documented and are passed around by word of mouth from one programmer to the next. For example, if you want to access a password-protected web page, you must do the following:

1. Concatenate the user name, a colon, and the password.
   Click here to view code image

   ```
   String input = username + ":" + password;
   ```

2. Compute the Base64 encoding of the resulting string. (The Base64 encoding encodes a sequence of bytes into a sequence of printable ASCII characters.)
   Click here to view code image

```
Base64.Encoder encoder = Base64.getEncoder();
String encoding = encoder.encodeToString(input.getBytes(StandardCharsets.UTF_8));
```

3. Call the `setRequestProperty` method with a name of `"Authorization"` and the value `"Basic " + encoding`.
   Click here to view code image

```
connection.setRequestProperty("Authorization", "Basic " + encoding);
```



Tip

You just saw how to access a password-protected web page. To access a password-protected file by FTP, use an entirely different method: Construct a URL of the form

Click here to view code image

```
ftp://username:password@ftp.yourserver.com/pub/file.txt
```

Once you call the `connect` method, you can query the response header information. First, let's see how to enumerate all response header fields. The implementors of this class felt a need to express their individuality by introducing yet another iteration protocol. The call

Click here to view code image

```
String key = connection.getHeaderFieldKey(n);
```

gets the `n` th key from the response header, where `n` starts from `1`! It returns `null` if `n` is zero or greater than the total number of header fields. There is no method to return the number of fields; you simply keep calling `getHeaderFieldKey` until you get `null`. Similarly, the call

Click here to view code image

```
String value = connection.getHeaderField(n);
```

returns the `n` th value.

The method `getHeaderFields` returns a `Map` of response header fields.

Click here to view code image

```
Map<String,List<String>> headerFields = connection.getHeaderFields();
```

Here is a set of response header fields from a typical HTTP request:

Click here to view code image

```
Date: Wed, 27 Aug 2008 00:15:48 GMT
Server: Apache/2.2.2 (Unix)
Last-Modified: Sun, 22 Jun 2008 20:53:38 GMT
Accept-Ranges: bytes
Content-Length: 4813
Connection: close
Content-Type: text/html
```



Note

You can obtain the response status line (such as `"HTTP/1.1 200 OK"`) as `connection.getHeaderField(0)` or `headerFields.get(null)`.

As a convenience, six methods query the values of the most common header types and convert them to numeric types when appropriate. **Table 4.1** shows these convenience methods. The methods with return type `long` return the number of seconds since January 1, 1970 GMT.

Table 4.1 Convenience Methods for Response Header Values

Key Name	Method Name	Return Type
`Date`	`getDate`	`long`

Key Name	Method Name	Return Type
Expires	getExpiration	long
Last-Modified	getLastModified	long
Content-Length	getContentLength	int
Content-Type	getContentType	String
Content-Encoding	getContentEncoding	String

The program in **Listing 4.6** lets you experiment with URL connections. Supply a URL and an optional user name and password on the command line when running the program, for example:

Click here to view code image

```
java urlConnection.URLConnectionTest http://www.yourserver.com user password
```

The program prints

- All keys and values of the header
- The return values of the six convenience methods in **Table 4.1**
- The first ten lines of the requested resource

Listing 4.6 urlConnection/URLConnectionTest.java

Click here to view code image

```
 1  package urlConnection;
 2
 3  import java.io.*;
 4  import java.net.*;
 5  import java.nio.charset.*;
 6  import java.util.*;
 7
 8  /** 9   * This program connects to an URL and displays the response header data and the first
10   * 10 lines of the requested data.
11   *
12   * Supply the URL and an optional username and password (for HTTP basic authentication) on
13   * the command line.
14   * @version 1.12 2018-03-17
15   * @author Cay Horstmann
16   */
17  public class URLConnectionTest
18  {
19     public static void main(String[] args)
20     {
21        try
22        {
23           String urlName;
24           if (args.length > 0) urlName = args[0];
25           else urlName = "http://horstmann.com";
26
27           var url = new URL(urlName);
28           URLConnection connection = url.openConnection();
```

```
29
30          // set username, password if specified on command line
31
32          if (args.length > 2)
33          {
34             String username = args[1];
35             String password = args[2];
36             String input = username + ":" + password;
37             Base64.Encoder encoder = Base64.getEncoder();
38             String encoding = encoder.encodeToString(input.getBytes(StandardCharsets.UTF_8))
39             connection.setRequestProperty("Authorization", "Basic " + encoding);
40          }
41
42          connection.connect();
43
44          // print header fields
45
46          Map<String, List<String>> headers = connection.getHeaderFields();
47          for (Map.Entry<String, List<String>> entry : headers.entrySet())
48          {
49             String key = entry.getKey();
50             for (String value : entry.getValue())
51                System.out.println(key + ": " + value);
52          }
53
54          // print convenience functions
55
56          System.out.println("----------");57            System.out.println("getContentType:
58          System.out.println("getContentLength: " + connection.getContentLength());
59          System.out.println("getContentEncoding: " + connection.getContentEncoding());
60          System.out.println("getDate: " + connection.getDate());
61          System.out.println("getExpiration: " + connection.getExpiration());
62          System.out.println("getLastModifed: " + connection.getLastModified());
```

```
63            System.out.println("----------");
64
65            String encoding = connection.getContentEncoding();
66            if (encoding == null) encoding = "UTF-8";
67            try (var in = new Scanner(connection.getInputStream(), encoding))
68            {
69               // print first ten lines of contents
70
71               for (int n = 1; in.hasNextLine() && n <= 10; n++)
72                   System.out.println(in.nextLine());
73               if (in.hasNextLine()) System.out.println(". . .");
74            }
75        }
76        catch (IOException e)
77        {
78           e.printStackTrace();
79        }
80     }
81  }
```

java.net.URL 1.0

- `InputStream openStream()`
  opens an input stream for reading the resource data.
- `URLConnection openConnection()`
  returns a `URLConnection` object that manages the connection to the resource.

java.net.URLConnection 1.0

- void setDoInput(boolean doInput)
- boolean getDoInput()
  If doInput is true, the user can receive input from this URLConnection.
- void setDoOutput(boolean doOutput)
- boolean getDoOutput()
  If doOutput is true, the user can send output to this URLConnection.
- void setIfModifiedSince(long time)
- long getIfModifiedSince()
  The ifModifiedSince property configures this URLConnection to fetch only data modified since a given time. The time is given in seconds since midnight, GMT, January 1, 1970.
- void setConnectTimeout(int timeout) 5.0
- int getConnectTimeout() 5.0
  set or get the timeout for the connection (in milliseconds). If the timeout has elapsed before a connection was established, the connect method of the associated input stream throws a SocketTimeoutException.
- void setReadTimeout(int timeout) 5.0
- int getReadTimeout() 5.0
  set or get the timeout for reading data (in milliseconds). If the timeout has elapsed before a read operation was successful, the read method throws a SocketTimeoutException.
- void setRequestProperty(String key, String value)
  sets a request header field.
- Map<String,List<String>> getRequestProperties() 1.4
  returns a map of request properties. All values for the same key are placed in a list.
- void connect()
  connects to the remote resource and retrieves the response header information.
- Map<String,List<String>> getHeaderFields() 1.4
  returns a map of response headers. All values for the same key are placed in a list.

- `String getHeaderFieldKey(int n)`
  gets the key for the `n` th response header field, or `null` if `n` is ≤ `0` or greater than the number of response header fields.
- `String getHeaderField(int n)`
  gets value of the `n` th response header field, or `null` if `n` is ≤ `0` or greater than the number of response header fields.
- `int getContentLength()`
  gets the content length if available, or `-1` if unknown.
- `String getContentType()`
  gets the content type, such as `text/plain` or `image/gif`.
- `String getContentEncoding()`
  gets the content encoding, such as `gzip`. This value is not commonly used, because the default `identity` encoding is not supposed to be specified with a `Content-Encoding` header.
- `long getDate()`
- `long getExpiration()`
- `long getLastModifed()`
  get the date of creation, expiration, and last modification of the resource. The dates are specified as seconds since midnight, GMT, January 1, 1970.
- `InputStream getInputStream()`
- `OutputStream getOutputStream()`
  return a stream for reading from the resource or writing to the resource.
- `Object getContent()`
  selects the appropriate content handler to read the resource data and convert it into an object. This method is not useful for reading standard types such as `text/plain` or `image/gif` unless you install your own content handler.

4.3.3 Posting Form Data

In the preceding section, you saw how to read data from a web server. Now we will show you how your programs can send data back to a web server and to programs that the web server invokes.

To send information from a web browser to the web server, a user fills out a *form*, like the one in **Figure 4.7**.

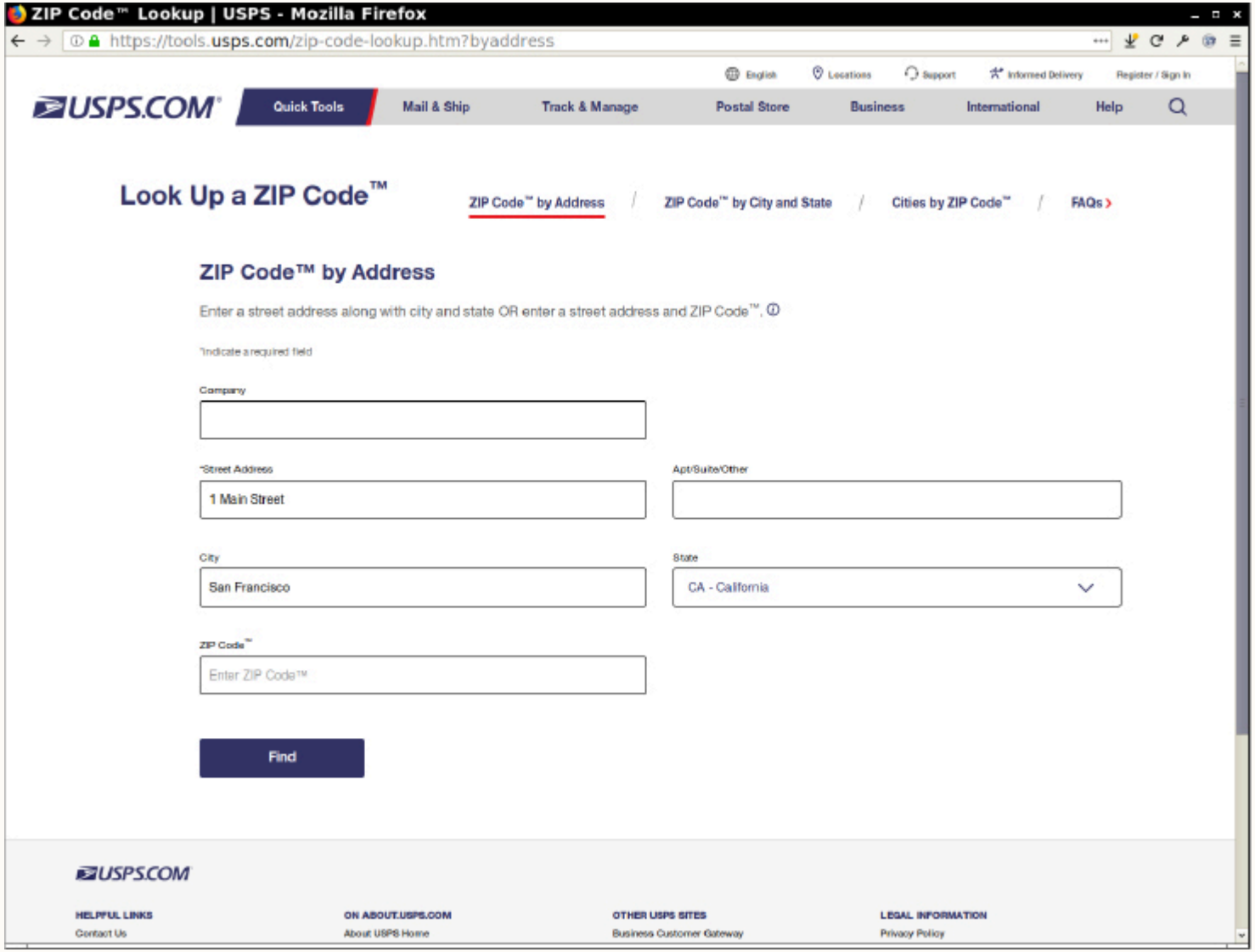


Figure 4.7 An HTML form

When the user clicks the Submit button, the text in the text fields and the settings of any checkboxes, radio buttons, and other input elements are sent back to the web server. The web server invokes a program that processes the user input.

Many technologies enable web servers to invoke programs. Among the best known ones are Java servlets, JavaServer Faces, Microsoft Active Server Pages (ASP), and Common Gateway Interface (CGI) scripts.

The server-side program processes the form data and produces another HTML page that the web server sends back to the browser. This sequence is illustrated in **Figure 4.8**. The response page can contain new information (for example, in an information-search program) or just an acknowledgment. The web browser then displays the response page.

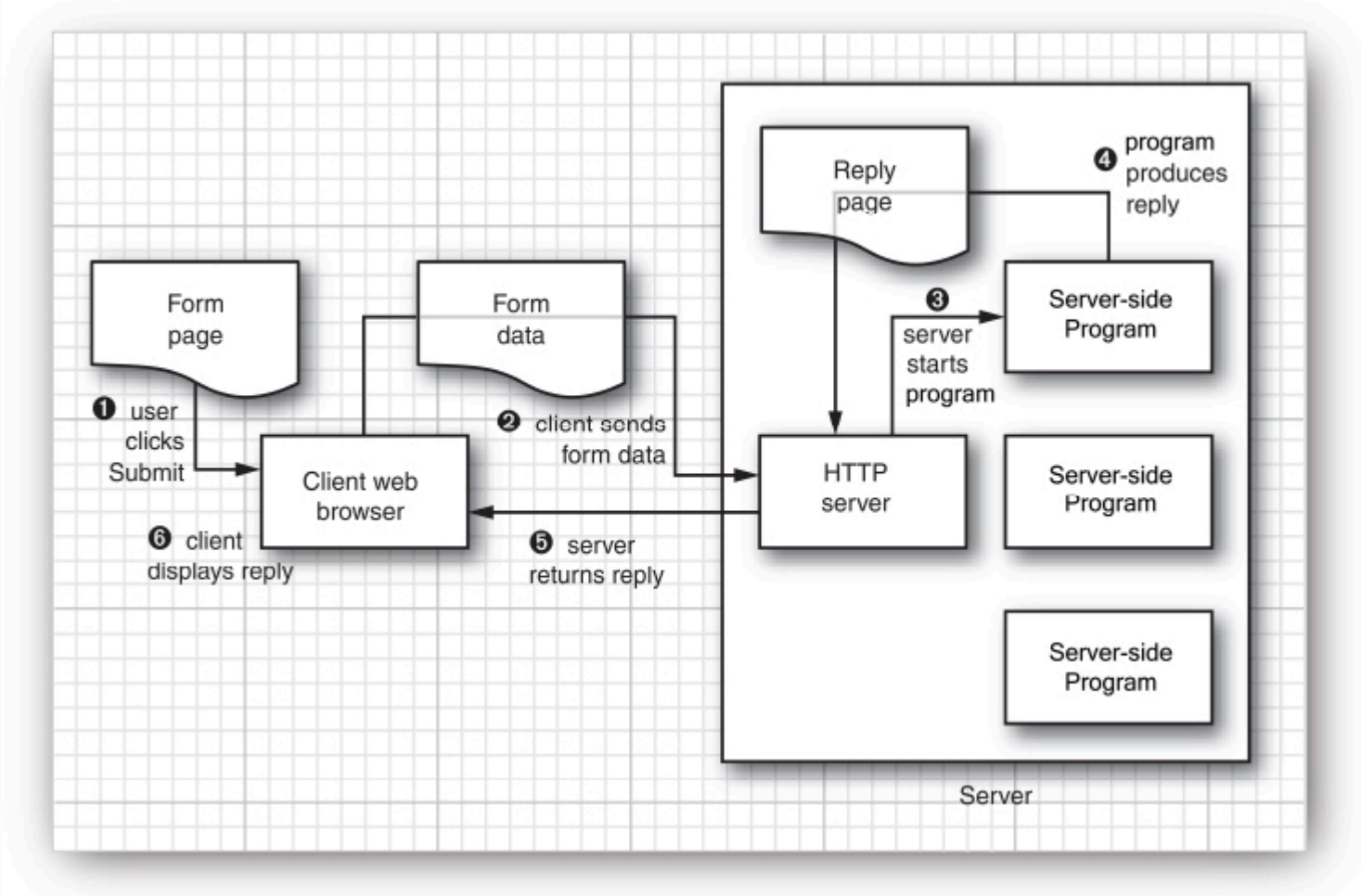


Figure 4.8 Data flow during execution of a server-side program

We do not discuss the implementation of server-side programs in this book. Our interest is merely in writing client programs that interact with existing server-side programs.

When form data are sent to a web server, it does not matter whether the data are interpreted by a servlet, a CGI script, or some other server-side technology. The client sends the data to the web server in a standard format, and the web server takes care of passing it on to the program that generates the response.

Two commands, called `GET` and `POST`, are commonly used to send information to a web server.

In the `GET` command, you simply attach query parameters to the end of the URL. The URL has the form

```
http://host/path?query
```

Each parameter has the form *name=value*. Parameters are separated by `&` characters. Parameter values are encoded using the *URL encoding* scheme, following these rules:

- Leave the characters `A` through `Z`, `a` through `z`, `0` through `9`, and `.-~_` unchanged.
- Replace all spaces with `+` characters.
- Encode all other characters into UTF-8 and encode each byte by a `%`, followed by a two-digit hexadecimal number.

For example, to transmit *San Francisco, CA*, you use `San+Francisco%2c+CA`, as the hexadecimal number `2c` is the UTF-8 code of the `','` character.

This encoding keeps any intermediate programs from messing with spaces and other special characters.

At the time of this writing, the Google Maps site (`www.google.com/maps`) accepts query parameters with names `q` and `hl` whose values are the location query and the human language of the response. To get a map of 1 Market Street in San Francisco, with a response in German, use the following URL:

Click here to view code image

```
http://www.google.com/maps?q=1+Market+Street+San+Francisco&hl=de
```

Very long query strings can look unattractive in browsers, and older browsers and proxies have a limit on the number of characters that you can include in a `GET` request. For that reason, a `POST` request is often used for forms with a lot of data. In a `POST` request, you do not attach parameters to a URL; instead, you get an output stream from the `URLConnection` and write name/value pairs to the output stream. You still have to URL-encode the values and separate them with `&` characters.

Let us look at this process in detail. To post data to a server-side program, first establish a `URLConnection`:

Click here to view code image

```
var url = new URL("http://host/path");
URLConnection connection = url.openConnection();
```

Then, call the `setDoOutput` method to set up the connection for output:

```
connection.setDoOutput(true);
```

Next, call `getOutputStream` to get a stream through which you can send data to the server. If you are sending text to the server, it is convenient to wrap that stream into a `PrintWriter`.

Click here to view code image

```
var out = new PrintWriter(connection.getOutputStream(), StandardCharsets.UTF_8);
```

Now you are ready to send data to the server:

Click here to view code image

```
out.print(name1 + "=" + URLEncoder.encode(value1, StandardCharsets.UTF_8) + "&");
out.print(name2 + "=" + URLEncoder.encode(value2, StandardCharsets.UTF_8));
```

Close the output stream:

```
out.close();
```

Finally, call `getInputStream` and read the server response.

Let's run through a practical example. The web site at **https://tools.usps.com/zip-code-lookup.htm?byaddress** contains a form to find the zip code for a street address (see **Figure 4.7** on p. **270**). To use this form in a Java program, you need to know the

URL and the parameters of the `POST` request.

You could get that information by looking at the HTML code of the form, but it is usually easier to "spy" on a request with a network monitor. Most browsers have a network monitor as part of their development toolkit. For example, **Figure 4.9** shows a screen capture of the Firefox network monitor when submitting data to our example web site. You can find out the submission URL as well as the parameter names and values.

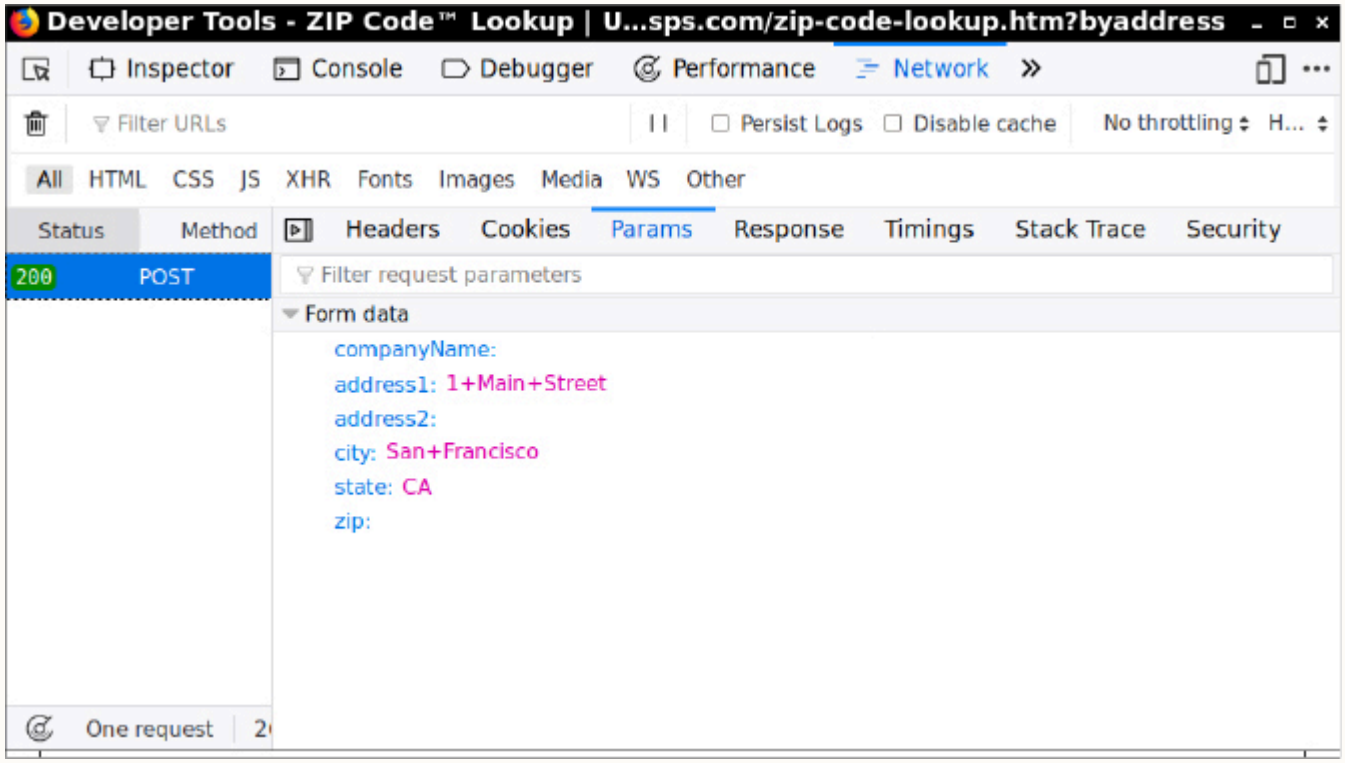


Figure 4.9 Monitoring the submission of a form

When posting form data, the HTTP header includes the content type:

Click here to view code image

```
Content-Type: application/x-www-form-urlencoded
```

You can also post data in other formats. For example, when sending data in JavaScript Object Notation (JSON), set the content type to `application/json`.

The header for a `POST` must also include the content length, for example

```
Content-Length: 124
```

The program in Listing 4.7 sends `POST` form data to any server-side program. Place the data into a `.properties` file such as the following:

Click here to view code image

```
url=https://tools.usps.com/tools/app/ziplookup/zipByAddress
User-Agent=HTTPie/0.9.2
address1=1 Market Street
address2=
city=San Francisco
state=CA
```

```
companyName=
. . .
```

The program removes the `url` and `User-Agent` entries and sends all others to the `doPost` method.

In the `doPost` method, we first open the connection and set the user agent. (The zip code service does not work with the default `User-Agent` request parameter which contains the string `Java`, perhaps because the postal service doesn't want to serve programmatic requests.)

Then we call `setDoOutput(true)`, and open the output stream. We then enumerate all keys and values. For each of them, we send the `key`, `=` character, `value`, and `&` separator character:

Click here to view code image

```
out.print(key);
out.print('=');
out.print(URLEncoder.encode(value, StandardCharsets.UTF_8));
if (more pairs) out.print('&');
```

When switching from writing to reading any part of the response, the actual interaction with the server happens. The `Content-Length` header is set to the size of the output. The `Content-Type` header is set to `application/x-www-form-urlencoded` unless a different content type was specified. The headers and data are sent to the server. Then the response headers and server response are read and can be queried. In our example program, this switch happens in the call to `connection.getContentEncoding()`.

There is one twist with reading the response. If a server-side error occurs, the call to `connection.getInputStream()` throws a `FileNotFoundException`. However, the server still sends an error page back to the browser (such as the ubiquitous "Error 404—page not found"). To capture this error page, call the `getErrorStream` method:

Click here to view code image

```
InputStream err = connection.getErrorStream();
```



Note

The `getErrorStream` method, as well as several other methods in this program, belong to the `HttpURLConnection` subclass of `URLConnection`. If you make a request to an URL that starts with `http://` or `https://`, you can cast the resulting connection object to `HttpURLConnection`.

When you send `POST` data to a server, it can happen that the server-side program responds with a *redirect*: a different URL that should be called to get the actual information. The server could do that because the information is available elsewhere, or to provide a bookmarkable URL. The `HttpURLConnection` class can handle redirects in most cases.



Note

If cookies need to be sent from one site to another in a redirect, you can configure the global cookie handler like this:

Click here to view code image

```
CookieHandler.setDefault(new CookieManager(null, CookiePolicy.ACCEPT_ALL));
```

Then cookies will be properly included in the redirect.

Even though redirects are usually automatically handled, there are some situations where you need to do them yourself. Automatic redirects between HTTP and HTTPS are not supported for security reasons. Redirects can also fail for more subtle reasons. For example, an earlier version of the zip code service used a redirect. Recall that we set the `User-Agent` request parameter so that the post office didn't think we made a request via the Java API. While it is possible to set the user agent to a different string in the initial request, that setting is not used in automatic redirects. Instead, automatic redirects always send a generic user agent string that contains the word `Java`.

In such situations, you can manually carry out the redirects. Before connecting the server, turn off automatic redirects:

Click here to view code image

```
connection.setInstanceFollowRedirects(false);
```

After making the request, get the response code:

Click here to view code image

```
int responseCode = connection.getResponseCode();
```

Check if it is one of

Click here to view code image

```
HttpURLConnection.HTTP_MOVED_PERM
HttpURLConnection.HTTP_MOVED_TEMP
HttpURLConnection.HTTP_SEE_OTHER
```

In that case, get the `Location` response header to obtain the URL for the redirect. Then disconnect and make another connection to the new URL:

Click here to view code image

```
String location = connection.getHeaderField("Location");
if (location != null)
{
  URL base = connection.getURL();
  connection.disconnect();
  connection = (HttpURLConnection) new URL(base, location).openConnection();
  . . .
}
```

The techniques that this program illustrates can be useful whenever you need to query information from an existing web site. Simply find out the parameters that you need to send, and then strip out the HTML tags and other unnecessary information from the reply.

Listing 4.7 post/PostTest.java

Click here to view code image

```
 1  package post;
 2
 3  import java.io.*;
 4  import java.net.*;
 5  import java.nio.charset.*; 6  import java.nio.file.*;
 7  import java.util.*;
 8
 9  /**
10   * This program demonstrates how to use the URLConnection class for a POST request.
11   * @version 1.43 2021-06-17
12   * @author Cay Horstmann
13   */
14  public class PostTest
15  {
16     public static void main(String[] args) throws IOException
17     {
18        String propsFilename = args.length > 0 ? args[0] : "post/post.properties";
19        var props = new Properties();
20        try (Reader in = Files.newBufferedReader(
21              Path.of(propsFilename), StandardCharsets.UTF_8))
22        {
23           props.load(in);
24        }
25        String urlString = props.remove("url").toString();
26        Object userAgent = props.remove("User-Agent");
27        Object redirects = props.remove("redirects");
28        CookieHandler.setDefault(new CookieManager(null, CookiePolicy.ACCEPT_ALL));
```

```
29        String result = doPost(new URL(urlString), props,
30           userAgent == null ? null : userAgent.toString(),
31           redirects == null ? -1 : Integer.parseInt(redirects.toString()));
32        System.out.println(result);
33     }
34
35     /**
36       * Do an HTTP POST.
37       * @param url the URL to post to
38       * @param nameValuePairs the query parameters
39       * @param userAgent the user agent to use, or null for the default user agent
40       * @param redirects the number of redirects to follow manually, or -1 for automatic
41       * redirects
42       * @return the data returned from the server
43       */
44      public static String doPost(URL url, Map<Object, Object> nameValuePairs, String userAgent
45           int redirects) throws IOException
46      {
47        var connection = (HttpURLConnection) url.openConnection();
48        if (userAgent != null)
49           connection.setRequestProperty("User-Agent", userAgent);
50
51        if (redirects >= 0)52            connection.setInstanceFollowRedirects(false);
53
54        connection.setDoOutput(true);
55
56        try (var out = new PrintWriter(connection.getOutputStream()))
57        {
58           boolean first = true;
59           for (Map.Entry<Object, Object> pair : nameValuePairs.entrySet())
60           {
61              if (first) first = false;
62              else out.print('&');
```

```
63              String name = pair.getKey().toString();
64              String value = pair.getValue().toString();
65              out.print(name);
66              out.print('=');
67              out.print(URLEncoder.encode(value, StandardCharsets.UTF_8));
68           }
69        }
70        String encoding = connection.getContentEncoding();
71        if (encoding == null) encoding = "UTF-8";
72
73        if (redirects > 0)
74        {
75           int responseCode = connection.getResponseCode();
76           if (responseCode == HttpURLConnection.HTTP_MOVED_PERM
77                 || responseCode == HttpURLConnection.HTTP_MOVED_TEMP
78                 || responseCode == HttpURLConnection.HTTP_SEE_OTHER)
79        {
80           String location = connection.getHeaderField("Location");
81           if (location != null)
82           {
83              URL base = connection.getURL();
84              connection.disconnect();
85              return doPost(new URL(base, location), nameValuePairs, userAgent,
86                    redirects - 1);
87           }
88        }
89     }
90     else if (redirects == 0)
91     {
92        throw new IOException("Too many redirects");
93     }
94
95     var response = new StringBuilder();
```

```
96      try (var in = new Scanner(connection.getInputStream(), encoding))
97      {
98         while (in.hasNextLine())
99         {
100              response.append(in.nextLine());
101              response.append("\n");
102        }
103     }
104     catch (IOException e)
105     {
106        InputStream err = connection.getErrorStream();
107        if (err == null) throw e;
108        try (var in = new Scanner(err))
109        {
110            response.append(in.nextLine());
111            response.append("\n");
112        }
113     }
114
115     return response.toString();
116   }
117 }
```

java.net.HttpURLConnection 1.0

- `InputStream getErrorStream()`
  returns a stream from which you can read web server error messages.

java.net.URLEncoder 1.0

- static String encode(String s, String encoding) 1.4
  returns the URL-encoded form of the string s, using the given character encoding scheme. (The recommended scheme is "UTF-8".) In URL encoding, the characters A–Z, a–z, 0–9, -_.~ are left unchanged. Space is encoded into +, and all other characters are encoded into sequences of encoded bytes of the form %*XY*, where 0x*XY* is the hexadecimal value of the byte.

java.net.URLDecoder 1.2

- static string decode(String s, String encoding) 1.4
  returns the decoding of the URL encoded string s under the given character encoding scheme.

4.4 The HTTP Client

The URLConnection class was designed before HTTP was the universal protocol of the Web. It provides support for a number of protocols, but its HTTP support is somewhat cumbersome. When the decision was made to support HTTP/2, it became clear that it would be best to provide a modern client interface instead of reworking the existing API. The HttpClient provides a more convenient API and HTTP/2 support. In Java 9 and 10, the API classes are located in the jdk.incubator.http package, to give the API a chance to evolve as a result of user feedback. As of Java 11, the HttpClient is in the java.net.http package.



Note

With Java 9 and 10, you need to run your program with the command-line option

Click here to view code image

```
--add-modules jdk.incubator.httpclient
```

The HTTP client API provides a simpler mechanism for connecting to a web server than the `URLConnection` class with its rather fussy set of stages.

4.4.1 The `HttpClient` Class

An `HttpClient` can issue requests and receive responses. You get a client by calling

Click here to view code image

```
HttpClient client = HttpClient.newHttpClient()
```

Alternatively, if you need to configure the client, you use a builder API, like this:

Click here to view code image

```
HttpClient client = HttpClient.newBuilder()
   .followRedirects(HttpClient.Redirect.ALWAYS)
   .build();
```

That is, you get a builder, call methods to customize the item that is going to be built, and then call the `build` method to finalize the building process. This is a common pattern for constructing immutable objects.

4.4.2 The `HttpRequest` class and Body Publishers

You also follow the builder pattern for formulating requests. Here is a `GET` request:

Click here to view code image

```
HttpRequest request = HttpRequest.newBuilder()
   .uri(new URI("http://horstmann.com"))
   .GET()
   .build();
```

The URI is the "uniform resource identifier" which is, when using HTTP, the same as a URL. However, in Java, the `URL` class has methods for actually opening a connection to a URL, whereas the `URI` class is only concerned with the syntax (scheme, host, port, path, query, fragment, and so on).

With a `POST` request, you need a "body publisher" that turns the request data into the data that is being posted. There are body publishers for strings, byte arrays, and files. For example, if your request is in JSON, you just provide the JSON string to a string body publisher.

Click here to view code image

```
HttpRequest request = HttpRequest.newBuilder()
   .uri(new URI(url))
```

```
    .header("Content-Type", "application/json")
    .POST(HttpRequest.BodyPublishers.ofString(jsonString))
    .build();
```

It is unfortunate that the API does not support the required formatting for common content types. The sample program in **Listing 4.8** provides body publishers for form data and file uploads.

Java 16 adds a builder for filtering the headers of an existing `HttpRequest`. You provide the request and a function that receives the header names and values, returning `true` for those that should be retained. For example, here we modify the content type:

Click here to view code image

```
HttpRequest request2 = HttpRequest.newBuilder(request,
    (name, value) -> !name.equalsIgnoreCase("Content-Type")) // Remove old content type
  .header("Content-Type", "application/xml") // Add new content type
  .build();
```

4.4.3 The `HttpResponse` Interface and Body Handlers

When sending the request, you have to tell the client how to handle the response. If you just want the body as a string, send the request with a `HttpResponse.BodyHandlers.ofString()`, like this:

Click here to view code image

```
HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());
```

`HttpResponse` is a generic interface whose type parameter denotes the type of the response body. You get the response body string simply as

Click here to view code image

```
String bodyString = response.body();
```

There are other response body handlers that get the response as a byte array or input stream. `BodyHandlers.ofFile(filePath)` yields a handler that saves the response to the given file, and `BodyHandlers.ofFileDownload(directoryPath)` saves the response in the given directory, using the file name from the `Content-Disposition` header. Finally, the handler obtained from `BodyHandlers.discarding()` simply discards the response.

Processing the contents of the response is not considered part of the API. For example, if you receive JSON data, you need a JSON library to parse the contents.

The `HttpResponse` object also yields the status code and the response headers.

Click here to view code image

```
int status = response.statusCode();
HttpHeaders responseHeaders = response.headers();
```

You can turn the `HttpHeaders` object into a map:

Click here to view code image

```
Map<String, List<String>> headerMap = responseHeaders.map();
```

The map values are lists since in HTTP, each key can have multiple values.

If you just want the value of a particular key, and you know that there won't be multiple values, call the `firstValue` method:

Click here to view code image

```
Optional<String> lastModified = headerMap.firstValue("Last-Modified");
```

You get the response value or an empty optional if none was supplied.

4.4.4 Asynchronous Processing

You can process the response asynchronously. When building the client, provide an executor:

Click here to view code image

```
ExecutorService executor = Executors.newCachedThreadPool();
HttpClient client = HttpClient.newBuilder().executor(executor).build();
```

Build a request and then invoke the `sendAsync` method on the client. You receive a `CompletableFuture<HttpResponse<T>>`, where `T` is the type of the body handler. Use the `CompletableFuture` API as described in **Chapter 12** of Volume I:

Click here to view code image

```
HttpRequest request = HttpRequest.newBuilder().uri(uri).GET().build();
client.sendAsync(request, HttpResponse.BodyHandlers.ofString())
  thenAccept(response -> . . .);
```



Tip

To enable logging for the `HttpClient`, add this line to `net.properties` in your JDK:

Click here to view code image

```
jdk.httpclient.HttpClient.log=all
```

Instead of `all`, you can specify a comma-separated list of `headers`, `requests`, `content`, `errors`, `ssl`, `trace`, and `frames`, optionally followed by `:control`, `:data`, `:window`, or `:all`. Don't use any spaces.

Then, set the logging level for the logger with name `jdk.httpclient.HttpClient` to `INFO`, for example by adding this line to the `logging.properties` file in your JDK:

Click here to view code image

```
jdk.httpclient.HttpClient.level=INFO
```

Listing 4.8 client/HttpClientTest.java

Click here to view code image

```
 1  package client;
 2
 3  import java.io.*;
 4  import java.math.*;
 5  import java.net.*;
 6  import java.nio.charset.*;
 7  import java.nio.file.*;
 8  import java.util.*;
 9
10  import java.net.http.*;
11  import java.net.http.HttpRequest.*;
12
13  class MoreBodyPublishers
14  {
15     public static BodyPublisher ofFormData(Map<Object, Object> data)
16     {
17        boolean first = true;
18        var builder = new StringBuilder();
19        for (Map.Entry<Object, Object> entry : data.entrySet())
20        {
21            if (first) first = false;
```

```
22            else builder.append("&");
23            builder.append(URLEncoder.encode(entry.getKey().toString(),
24               StandardCharsets.UTF_8));
25            builder.append("=");
26            builder.append(URLEncoder.encode(entry.getValue().toString(),
27               StandardCharsets.UTF_8));
28         }
29         return BodyPublishers.ofString(builder.toString());
30      }
31
32      private static byte[] bytes(String s) { return s.getBytes(StandardCharsets.UTF_8); }
3334      public static BodyPublisher ofMimeMultipartData(Map<Object, Object> data, String boundar
35            throws IOException
36      {
37         var byteArrays = new ArrayList<byte[]>();
38         byte[] separator = bytes("--" + boundary + "\nContent-Disposition: form-data; name=");
39         for (Map.Entry<Object, Object> entry : data.entrySet())
40         {
41            byteArrays.add(separator);
42
43            if (entry.getValue() instanceof Path path)
44            {
45               String mimeType = Files.probeContentType(path);
46               byteArrays.add(bytes("\"" + entry.getKey() + "\"; filename=\""
47               + path.getFileName() + "\"\nContent-Type: " + mimeType + "\n\n"));
48               byteArrays.add(Files.readAllBytes(path));
49            }
50            else
51               byteArrays.add(bytes("\"" + entry.getKey() + "\"\n\n" + entry.getValue() + "\n"));
52         }
53         byteArrays.add(bytes("--" + boundary + "--"));
54         return BodyPublishers.ofByteArrays(byteArrays);
55      }
```

```
56
57    public static BodyPublisher ofSimpleJSON(Map<Object, Object> data)
58    {
59       var builder = new StringBuilder();
60       builder.append("{");
61       var first = true;
62       for (Map.Entry<Object, Object> entry : data.entrySet())
63       {
64          if (first) first = false;
65          else
66             builder.append(",");
67          builder.append(jsonEscape(entry.getKey().toString())).append(": ")
68             .append(jsonEscape(entry.getValue().toString()));
69       }
70       builder.append("}");
71       return BodyPublishers.ofString(builder.toString());
72    }
73
74    private static Map<Character, String> replacements = Map.of('\b', "\\b", '\f', "\\f",
75       '\n', "\\n", '\r', "\\r", '\t', "\\t", '"', "\\\"", '\\', "\\\\");
76
77    private static StringBuilder jsonEscape(String str)
78    {
79       var result = new StringBuilder("\"");
80       for (int i = 0; i < str.length(); i++)
81       {
82           char ch = str.charAt(i);83           String replacement = replacements.get(ch);
84           if (replacement == null) result.append(ch);
85           else result.append(replacement);
86       }
87       result.append("\"");
88       return result;
89    }
```

```
90  }
91
92  public class HttpClientTest
93  {
94      public static void main(String[] args)
95            throws IOException, URISyntaxException, InterruptedException
96      {
97         System.setProperty("jdk.httpclient.HttpClient.log", "headers,errors");
98         String propsFilename = args.length > 0 ? args[0] : "client/post.properties";
99         Path propsPath = Path.of(propsFilename);
100         var props = new Properties();101         try (Reader in = Files.newBufferedReader(pro
102         {
103            props.load(in);
104         }
105         String urlString = "" + props.remove("url");
106         String contentType = "" + props.remove("Content-Type");
107         if (contentType.equals("multipart/form-data"))
108         {
109            var generator = new Random();
110            String boundary = new BigInteger(256, generator).toString();
111            contentType += ";boundary=" + boundary;
112            props.replaceAll((k, v) ->
113               v.toString().startsWith("file://")
114                  ? propsPath.getParent().resolve(Path.of(v.toString().substring(7)))
115                  : v);
116         }
117         String result = doPost(urlString, contentType, props);
118         System.out.println(result);
119      }
120
121      public static String doPost(String url, String contentType, Map<Object, Object> data)
122            throws IOException, URISyntaxException, InterruptedException
123      {
```

```
124         HttpClient client = HttpClient.newBuilder()
125           .followRedirects(HttpClient.Redirect.ALWAYS).build();
126
127         BodyPublisher publisher = null;
128         if (contentType.startsWith("multipart/form-data"))
129         {
130            String boundary = contentType.substring(contentType.lastIndexOf("=") + 1);131
132         }
133         else if (contentType.equals("application/x-www-form-urlencoded"))
134            publisher = MoreBodyPublishers.ofFormData(data);
135         else
136         {
137            contentType = "application/json";
138            publisher = MoreBodyPublishers.ofSimpleJSON(data);
139         }
140
141         HttpRequest request = HttpRequest.newBuilder()
142            .uri(new URI(url))
143            .header("Content-Type", contentType)
144            .POST(publisher)
145            .build();
146         HttpResponse<String> response
147            = client.send(request, HttpResponse.BodyHandlers.ofString());
148         return response.body();
149      }
150   }
```

java.net.http.HttpClient 11

- static HttpClient newHttpClient()

yields an `HttpClient` with a default configuration.

- `static HttpClient.Builder newBuilder()`
  yields a builder for building an `HttpClient`.
- `<T> HttpResponse<T> send(HttpRequest request, HttpResponse.BodyHandler<T> responseBodyHandler)`
- `<T> CompletableFuture<HttpResponse<T>> sendAsync(HttpRequest request, HttpResponse.BodyHandler<T> responseBodyHandler)`
  make a synchronous or asynchronous request and process the response body with the given handler.

`java.net.http.HttpClient.Builder` 11

- `HttpClient build()`
  yields an `HttpClient` with the properties configured by this builder.
- `HttpClient.Builder followRedirects(HttpClient.Redirect policy)`
  sets the redirect policy to one of the values `ALWAYS`, `NEVER`, or `NORMAL` (only refuse redirects from HTTPS to HTTP) of the `HttpClient.Redirect` enumeration.
- `HttpClient.Builder executor(Executor executor)`
  sets the executor for asynchronous requests.

`java.net.http.HttpRequest` 11

- `HttpRequest.Builder newBuilder()`
  returns a builder for building an `HttpRequest`.
- `HttpRequest.Builder newBuilder(HttpRequest request, BiPredicate<String,String> filter)` 16

returns a builder for building a request with the same properties as the provided one, with only the headers that are accepted by the filter.

---

java.net.http.HttpRequest.Builder 11

- `HttpRequest build()`
  yields an `HttpRequest` with the properties configured by this builder.
- `HttpRequest.Builder uri(URI uri)`
  sets the URI for this request.
- `HttpRequest.Builder header(String name, String value)`
  sets a request header for this request.
- `HttpRequest.Builder GET()`
- `HttpRequest.Builder DELETE()`
- `HttpRequest.Builder POST(HttpRequest.BodyPublisher bodyPublisher)`
- `HttpRequest.Builder PUT(HttpRequest.BodyPublisher bodyPublisher)`
  set the request method and body for this request.

---

java.net.http.HttpResponse<T> 11

- `T body()`
  yields the body of this response.
- `int statusCode()`
  yields the status code for this response.
- `HttpHeaders headers()`

yields the response headers.

---

java.net.http.HttpHeaders 11

- `Map<String,List<String>> map()`
  yields a `Map` of these headers.
- `Optional<String> firstValue(String name)`
  The first value with the given name in these headers, if present.

4.5 Sending E-Mail

In the past, it was simple to write a program that sends e-mail by making a socket connection to port 25, the SMTP port, of a mail server. The Simple Mail Transport Protocol (SMTP) describes the format for e-mail messages. Once you are connected to the server, send a mail header (in the SMTP format, which is easy to generate), followed by the mail message.

Here are the details:

1. Open a socket to your host.
   Click here to view code image

```
var s = new Socket("mail.yourserver.com", 25); // 25 is SMTP
var out = new PrintWriter(s.getOutputStream(), StandardCharsets.UTF_8);
```

2. Send the following information to the print stream:
   Click here to view code image

`HELO` *sending host* `MAIL FROM:` *sender e-mail address* `RCPT TO:` *recipient e-mail address* `DATA` `Subject:` *subject (blank line) mail message (any number of lines)* . `QUIT`

The SMTP specification (RFC 821) states that lines must be terminated with `\r` followed by `\n`.

It used to be that SMTP servers were often willing to route e-mail from anyone. However, in these days of spam floods, most servers have built-in checks and only accept requests from users or IP address ranges that they trust. Authentication usually happens over secure socket connections.

Implementing these authentication schemes manually would be very tedious. Instead, we will show you how to use the JavaMail API to send e-mail from a Java program.

Download JavaMail from **www.oracle.com/technetwork/java/javamail** and unzip it somewhere on your hard disk.

To use JavaMail, you need to set up some properties that depend on your mail server. For example, with GMail, you use

Click here to view code image

```
mail.transport.protocol=smtps
mail.smtps.auth=true
mail.smtps.host=smtp.gmail.com
mail.smtps.user=accountname@gmail.com
```

Our sample program reads these from a property file.

For security reasons, we don't put the password into the property file but instead prompt for it.

Read in the property file, then get a mail session like this:

Click here to view code image

```
Session mailSession = Session.getDefaultInstance(props);
```

Make a message with the desired sender, recipient, subject, and message text:

Click here to view code image

```
var message = new MimeMessage(mailSession);
message.setFrom(new InternetAddress(from));
message.addRecipient(RecipientType.TO, new InternetAddress(to));
message.setSubject(subject);
message.setText(builder.toString());
```

Then send it off:

Click here to view code image

```
Transport tr = mailSession.getTransport();
tr.connect(null, password);
tr.sendMessage(message, message.getAllRecipients());
tr.close();
```

The program in **Listing 4.9** reads the message from a text file of the format

Sender
Recipient
Subject
Message text (any number of lines)

To run the program, download the JavaMail implementation from `https://javaee.github.io/javamail` . You also need the JAR file for the Java Activation Framework—get it from `https://www.oracle.com/java/technologies/downloads.html` or search on Maven Central. Then run

Click here to view code image

```
java -classpath .:javax.mail.jar:activation-1.1.1.jar path/to/message.txt
```

At the time of this writing, GMail does not check the veracity of the information—you can supply any sender you like. (Keep this in mind the next time you get an e-mail message from `president@whitehouse.gov` inviting you to a black-tie affair on the front lawn.)



Tip

If you can't figure out why your mail connection isn't working, call

```
mailSession.setDebug(true);
```

and check out the messages. Also, the JavaMail API FAQ has some useful debugging hints.

Listing 4.9 mail/MailTest.java

Click here to view code image

```
 1  package mail;
 2
 3  import java.io.*;
 4  import java.nio.charset.*;
 5  import java.nio.file.*;
 6  import java.util.*;
 7  import javax.mail.*;
 8  import javax.mail.internet.*;
 9  import javax.mail.internet.MimeMessage.RecipientType;
10
11  /**
12   * This program shows how to use JavaMail to send mail messages.
13   * @author Cay Horstmann
14   * @version 1.02 2021-06-17
15   */
16  public class MailTest
17  {
18     public static void main(String[] args) throws MessagingException, IOException
19     {
20        var props = new Properties();
21        try (Reader in = Files.newBufferedReader(Path.of("mail",
22              "mail.properties"), StandardCharsets.UTF_8))
23        {
24           props.load(in);
```

```
25        }
26        List<String> lines = Files.readAllLines(Path.of(args[0]), StandardCharsets.UTF_8);
27
28        String from = lines.get(0);
29        String to = lines.get(1);
30        String subject = lines.get(2);
31
32        var builder = new StringBuilder();
33        for (int i = 3; i < lines.size(); i++)
34        {
35           builder.append(lines.get(i));
36           builder.append("\n");
37        }
38
39        Console console = System.console();
40        var password = new String(console.readPassword("Password: "));
41
42        Session mailSession = Session.getDefaultInstance(props);
43        // mailSession.setDebug(true);
44        var message = new MimeMessage(mailSession);
45        message.setFrom(new InternetAddress(from));
46        message.addRecipient(RecipientType.TO, new InternetAddress(to));
47        message.setSubject(subject);48        message.setText(builder.toString());
49        Transport tr = mailSession.getTransport();
50        try
51        {
52           tr.connect(null, password);
53           tr.sendMessage(message, message.getAllRecipients());
54        }
55        finally
56        {
57           tr.close();
58        }
```

```
59     }
60 }
```

In this chapter, you have seen how to write network clients and servers in Java and how to harvest information from web servers. The next chapter covers database connectivity. You will learn how to work with relational databases in Java, using the JDBC API.

Previous chapter
< Chapter 3. XML

Next chapter
Chapter 5. Database Programming >